

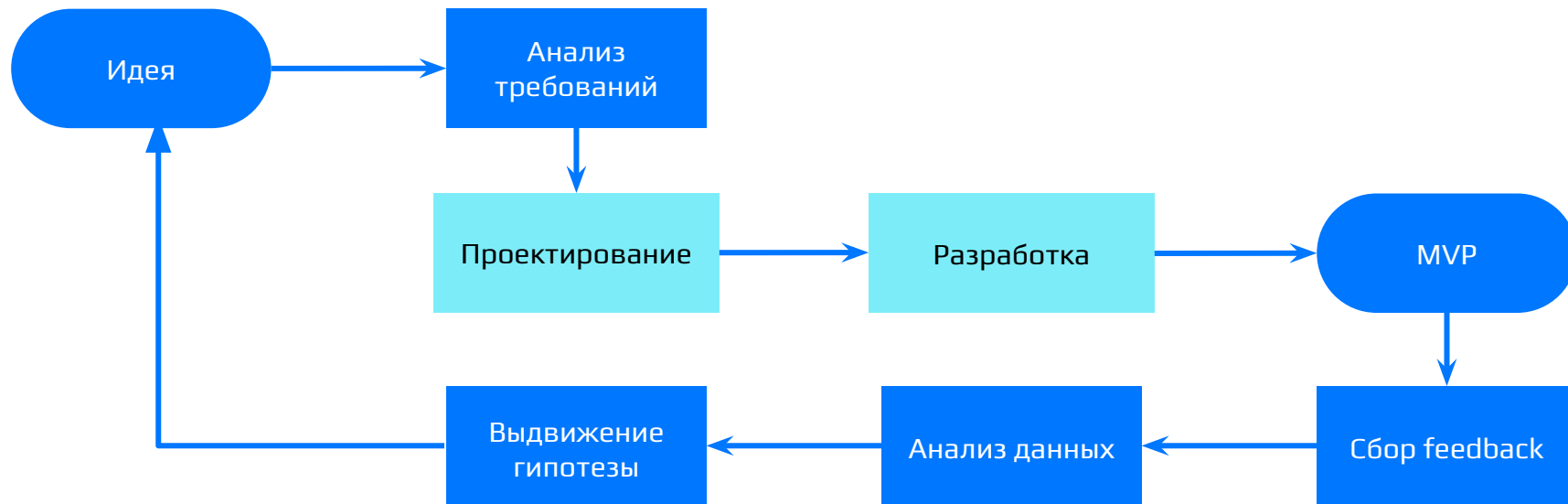


От тестирования до продуктовых гипотез

Олег Гибадулин



Этапы

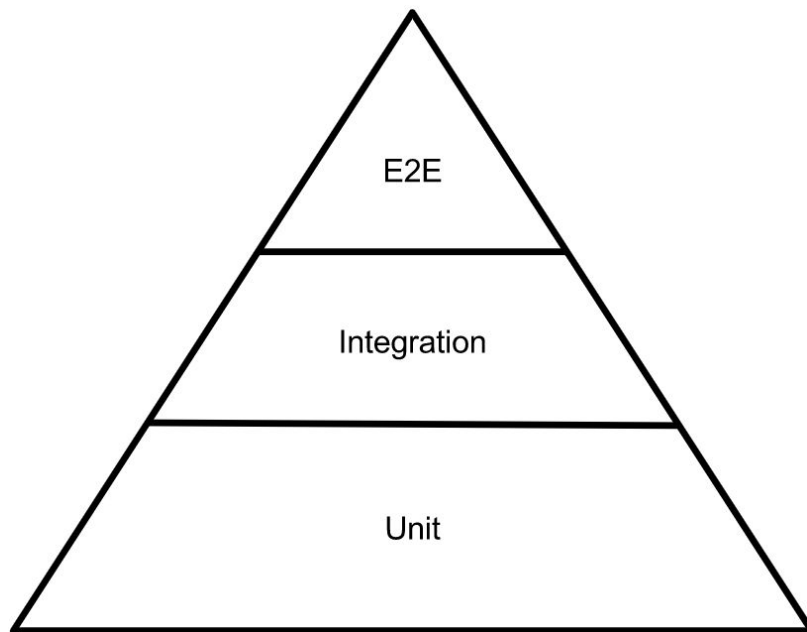


Тестирование

Тестирование - это не про поиск багов

- Цель тестов - управление рисками.
- Тесты дают уверенность при рефакторинге.
- Ручное тестирование не масштабируется: чем больше проект, тем дольше релиз.

Пирамида тестирования



Given-When-Then



Атомарность

Тесты не должны зависеть друг от друга и от порядка их запуска.

Если Тест А проходит, когда запускается первым, но падает, если запускается после Теста Б - это проблема. Значит, Тест Б "намусорил" в глобальном состоянии (не закрыл файл, оставил записи в базе данных, изменил глобальную переменную).

Жизненный цикл теста

Setup: Код, который выполняется до каждого теста. Здесь мы готовим "чистую комнату": создаем свежие экземпляры объектов, чистим базу, инициализируем стабы.

Teardown: Код, который выполняется после каждого теста. Здесь мы "убираем за собой": закрываем коннекты к БД, удаляем временные файлы.

Тестируемость кода

Плохой код: Функция вычисляет скидку и внутри себя делает SQL-запрос к базе данных, чтобы узнать статус пользователя. Чтобы протестировать эту функцию, вам нужно поднять реальную БД, наполнить её данными, выполнить тест и очистить БД. Это медленно и нестабильно.

Хороший код (Dependency Injection): Функция принимает статус пользователя как аргумент (или принимает интерфейс UserRepository). Теперь в тесте вы можете передать «фейкового» пользователя без всякой БД.

Test Doubles

- **Dummy:** Пустышка. Передается в функцию просто чтобы компилятор не ругался (например, обязательный параметр, который не используется в конкретном сценарии).
- **Stub:** Заглушка. Возвращает жестко заданный ответ. Пример: на вызов `getUser()` всегда возвращает `{name: "Ivan"}`. Мы не проверяем, как часто его вызывали, нам важен только результат.
- **Mock:** Умная заглушка, которая проверяет поведение. Пример: мы проверяем, что при ошибке оплаты функция ровно один раз вызвала `logger.logError()`.
- **Fake:** Работающая, но упрощенная реализация. Пример: In-Memory база данных (SQLite) вместо реального PostgreSQL на сервере.

Code Coverage

- Line Coverage: Какой процент строк кода был выполнен во время запуска тестов.
- Branch Coverage: Прошли ли тесты по всем ветвям if / else. Это самая важная метрика из стандартных. Если у вас есть if ($x > 0$), тесты должны запустить этот код дважды: когда $x = 1$ и когда $x = -1$.

Почему Code Coverage врет?

Вы можете написать тест, который вызовет все функции в вашем проекте, но вы забудете написать слово `assert` в конце. Покрытие будет 100%, но тест ничего не проверяет. Код может выдавать неверный результат, а тест будет зеленым. Количественная метрика создает ложное чувство безопасности.

Иллюзия покрытия и Flaky-тесты

- Покрытие в 100% не гарантирует отсутствие багов.
- Flaky test (тест, который падает случайно) - это хуже, чем отсутствие теста. Он убивает доверие команды к пайплайну сборки.

Другие важные метрики

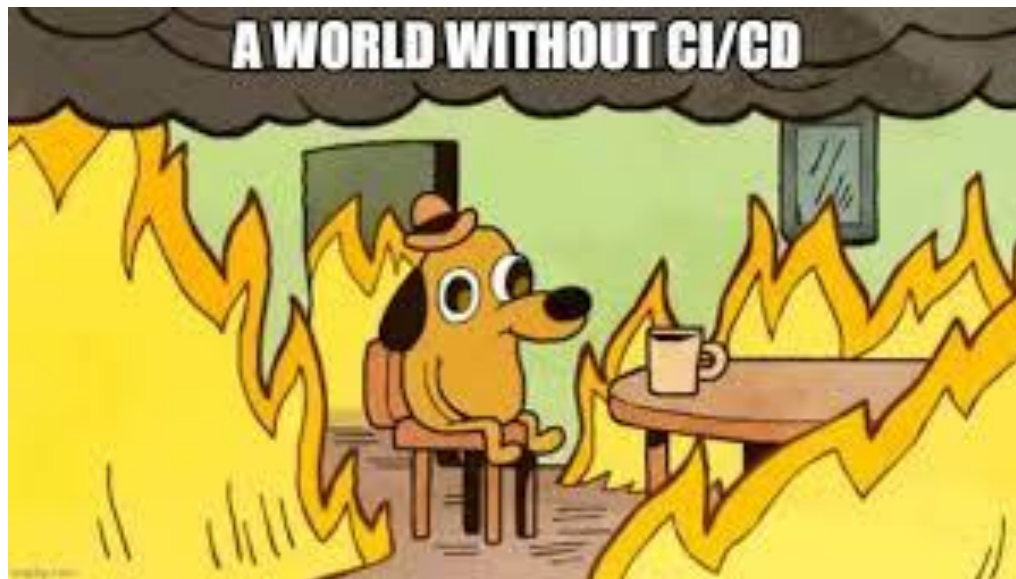
- **Flakiness rate** (Процент моргающих тестов): Сколько раз тест падал, а при автоматическом перезапуске проходил успешно. Высокий показатель говорит о проблемах с изоляцией тестов (тесты мешают друг другу) или таймингами (сетевые задержки).
- **Test Suite Execution Time**: Время прохождения всех тестов. Если тесты идут дольше 10-15 минут, разработчики начинают реже коммитить код, процесс интеграции (CI) тормозится.

Вопросы?



CI/CD

Зачем?



Continuous Integration

- Интеграция кода происходит несколько раз в день небольшими порциями (избегаем Merge Hell).
- При каждом PR запускается изолированная среда (Runner).
- Ветка main заблокирована для прямого пуша: слить код можно только если робот дал зеленый свет.



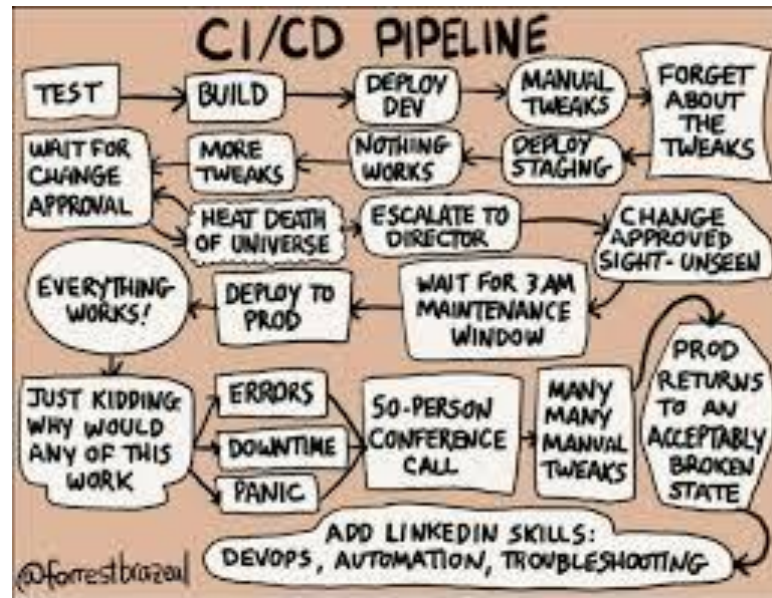
Svetozar

@sgremyachikh

- Как вы понимаете CI/CD
- Сияй, сиди)
- Что?
- Что?

Что происходит под капотом

- **Lint:** Проверка синтаксиса и стиля кода (работает за секунды).
- **Test:** Запуск Unit и Integration тестов (наши стабы и моки в действии).
- **Build:** Компиляция артефакта (создание .ipa/.apk для мобилок или Docker-образа для бэкенда).
- **Upload:** Отправка артефакта тестировщикам (в TestFlight, Firebase App Distribution или на Staging сервер).



Continuous Delivery

Пайплайн собирает готовый артефакт (например, скомпилированный .apk файл для Android или Docker-образ) и кладет его на "склад" (Реестр артефактов). Код готов к релизу в любую секунду, но сам релиз происходит по нажатию кнопки человеком (релиз-менеджером).

Continuous Deployment

Продолжение Delivery. Если тесты прошли успешно, система автоматически, без участия человека, выкатывает новый код на боевые сервера (Production) к реальным пользователям.

Delivery vs Deployment

Continuous Delivery: Сборка полностью готова к релизу и лежит на «складе». Выпуск в продакшен требует нажатия кнопки человеком (iOS, Android).

Continuous Deployment: Код автоматически улетает прямо на сервера к пользователям (веб, бэкенд).

Инструментарий индустрии

- Вся магия CI/CD описывается обычным текстовым файлом, который лежит в том же репозитории, что и код.
- Оркестраторы. Это системы, которые слушают ваш Git-репозиторий и запускают пайплайны. Они ничего не знают про ваш код, они просто выполняют .yaml файлы.
- GitHub Actions / GitLab CI / Jenkins

Backend

- Среда сборки: Docker (Linux).
- Arteфакт сборки: Docker Image.
- Деплой (CD): Кубернетес (Kubernetes / K8s) или просто Docker Compose на сервере.

Frontend

- Среда сборки: Docker.
- Arteфакт сборки: Папка dist (набор статических файлов HTML/CSS/JS).
- Деплой (CD): Файлы заливаются на S3-бакет и раздаются через Nginx.

Mobile

- Среда сборки: macOS Runner (для iOS) / Linux Runner (для Android).
- Компилятор: xcodebuild (iOS), gradle (Android).
- Магия автоматизации: Fastlane - скрипты на Ruby, которые сами делают скриншоты, скачивают сертификаты для подписи (Match) и инкрементят номер сборки.
- Arteфакт сборки: .ipa или .apk / .aab.
- Деплой (CD): TestFlight (бета-тестеры) → App Store Connect (прод). Firebase App Distribution (для Android).

Вопросы?



Метрики

— Как вы получаете такие крутые показатели?

— Мы неправильно считаем.



Код в продакшене. Что дальше?

- Релиз — это не финал, это начало сбора данных.
- Без метрик любая жалоба пользователя — это загадка, а любая новая фича — гадание на кофейной гуще.

Технические метрики: Здоровье кода

- Crash-free (99.9%): Главный показатель качества релиза.
- ANR (Зависания): Хуже, чем краш.
- Latency / Time to Interactive: За сколько секунд прогружается интерфейс.

Инструмент: Sentry, Firebase Crashlytics, Datadog, Prometheus + Grafana.

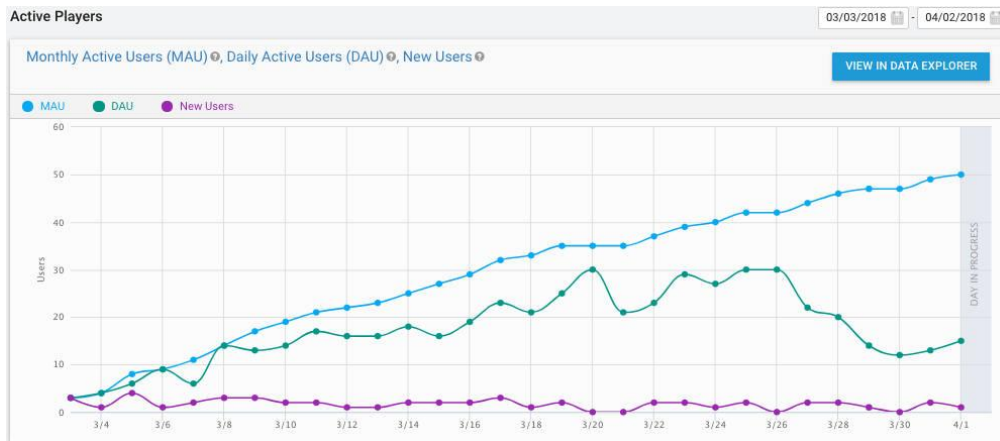


Продуктовые метрики: Здоровье бизнеса

- **DAU / MAU** (Daily/Monthly Active Users): Сколько уникальных пользователей заходит в приложение каждый день/месяц.
- **Retention Rate** (Возвращаемость): Показывает, какой процент людей вернулся в приложение на 1-й, 7-й и 30-й день после установки.
- **Воронка конверсии** (Funnel): Пользователь не покупает товар по щелчку. Он проходит этапы: Открыл приложение → Зашел в каталог → Положил в корзину → Ввел карту → Купил. Воронка показывает, на каком именно этапе отваливается больше всего людей.

Продуктовые метрики. Инструменты

- Amplitude
- Mixpanel
- AppMetrica
- Google Analytics



Вопросы?



Гипотезы

Продуктовые метрики: Здоровье бизнеса

- Разработка фич «по наитию» - путь к провалу. Каждая задача должна проверять гипотезу.
- Быстрые итерации: чем короче цикл HADI, тем быстрее продукт находит то, что нужно рынку (Product-Market Fit).

HADE-циклы

1 Hypothesis

Гипотеза

Формулировка гипотезы, которая предполагает, что изменение в процессе может улучшить результат.

2 Action

Действие

Внедрение конкретных действий для проверки гипотезы.

3 Insight

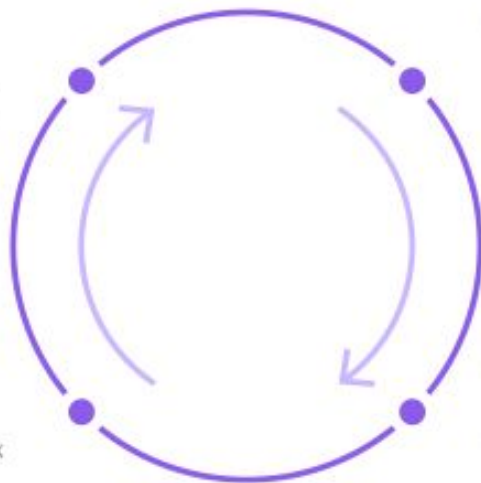
Инсайт

Анализ полученных данных и выработка выводов для дальнейших шагов.

4 Data

Данные

Сбор данных о результатах изменений.



Как формулировать задачи

- **Плохая задача:** «Давайте добавим оплату через Apple Pay/Google Pay». (Почему добавляем? Что ждем в итоге? Непонятно).
- **Хорошая гипотеза:** «**Если** мы добавим Apple/Google Pay, **то конверсия** в успешную оплату (метрика) вырастет на **5%**, **потому что** пользователям не придется вручную вводить данные карты (обоснование)».

Эксперименты на пользователях (A/B Tests)

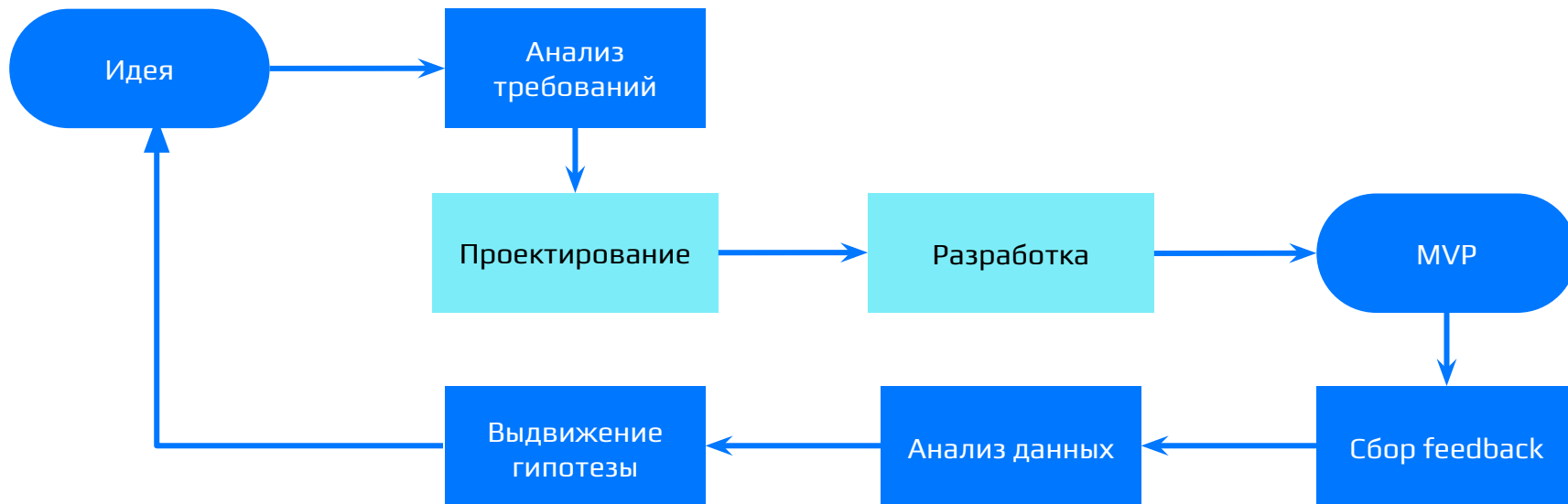
- Минимизируем риски: если гипотеза оказалась провальной (у Группы В упали продажи), мы просто отключаем тест на сервере (меняем Feature Toggle), не выпуская новый релиз.
- Требуется статистическая значимость (достаточное количество пользователей).

Замыкание петли (SDLC)

- Инсайты из HADI-цикла порождают новые задачи (Plan).
- Разработчик пишет код для новой гипотезы (Code).
- Код покрывается тестами для надежности (Test).
- CI/CD автоматически доставляет его пользователям (Deploy).
- Мы собираем новые метрики (Monitor).
- И цикл начинается снова.



Замыкание петли (SDLC)



Вопросы?



Практика

Все задания выданы



Дедлайн по сдаче проекта - 6 мая



Экзамен

Тайминг: 20 минут

Цель: защитить проект

Задача: быть готовым ответить на теоретические вопросы по курсу. Например, “чем канбан отличается от скрама”, “что такое BDD”, “как приоритезировать задачу по RICE”

Слоты: 6 мая, 13 мая, 20 мая

[Ведомость](#)

Спасибо за внимание!

Не забудьте оставить отзыв :)

